

1 Các thuật toán khai phá tập hạng mục thường xuyên

- Các phương pháp phát triển mẫu dựa vào hậu tố đệ quy
- Các phương pháp khác
- Một số thuật toán meta có ích

2 Phân tích gom nhóm

- Gom nhóm
- Chọn đặc trưng để gom nhóm

Các phương pháp phát triển mẫu dựa vào hậu tố đệ quy

- Các cây đánh số được xây dựng bằng cách mở rộng các tiền tố (prefix) của các tập hạng mục với thứ tự từ điển.
- Tuy nhiên, chúng ta cũng có thể biểu diễn một số lớp phương pháp khám phá tập danh mục theo cách đệ quy với khám phá theo hậu tố (suffix).

Các phương pháp phát triển mẫu dựa vào hậu tố đệ quy

- Các phương pháp phát triển mẫu theo hậu tố đệ quy thường được hiểu với cấu trúc dữ liệu cây mẫu thường xuyên (FP-tree).
- Cấu trúc cây mẫu thường xuyên này giúp chúng ta thực hiện các thuật toán khám phá mẫu đệ quy một cách tiết kiệm.
- Tuy nhiên, các thuật toán này cũng có thể được thực hiện với mảng (array) và pointer.

Thuật toán Recursive Suffix Growth

Algorithm *RecursiveSuffixGrowth*(Transactions in terms of frequent 1-items: $\mathcal{T}$, Minimum Support: $minsup$, Current Suffix: P)

begin

for each item i in $\mathcal{T}$ **do begin**

report itemset $P_i = \{i\} \cup P$ as frequent;

 Extract all transactions $\mathcal{T}_i$ from $\mathcal{T}$ containing item i ;

 Remove all items from $\mathcal{T}_i$ that are lexicographically $\geq i$;

 Remove all infrequent items from $\mathcal{T}_i$;

if ($\mathcal{T}_i \neq \phi$) **then** *RecursiveSuffixGrowth*($\mathcal{T}_i, minsup, P_i$);

end

end

Figure 4.8: Generic recursive suffix growth on transaction database expressed in terms of frequent 1-items

Thực hiện với mảng nhưng không có pointer

- Các tập giao dịch và giao dịch điều kiện trong thuật toán có thể được biểu diễn bằng các mảng.
- Các vòng lặp for khi sử dụng mảng rất **tốn kém tính toán**.
- Để có trade off tốt hơn giữa yêu cầu **tính toán** và yêu cầu **lưu trữ**, chúng ta có thể sử dụng thêm pointer (cần nhiều lưu trữ hơn nhưng ít tính toán hơn).

Thực hiện với pointer nhưng không có cây mẫu thường xuyên

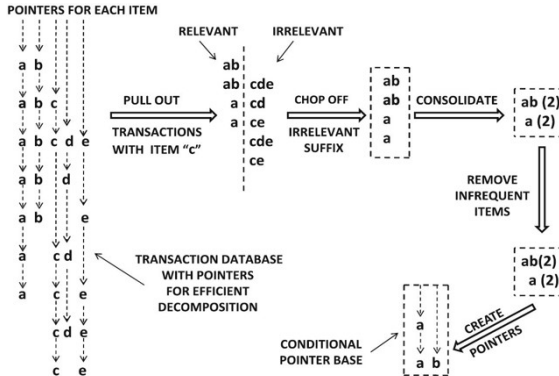


Figure 4.9: Illustration of recursive pattern growth with pointers and no FP-Tree

Thực hiện với pointer nhưng không có cây mẫu thường xuyên

Algorithm *RecursiveGrowthPointers*(Transactions in terms of frequent 1-items: $\mathcal{T}$,
Minimum Support: $minsup$, Current Suffix: P)

begin

 for each item i in $\mathcal{T}$ do begin

 report itemset $P_i = \{i\} \cup P$ as frequent;

 Use pointers to extract all transactions $\mathcal{T}_i$
 from $\mathcal{T}$ containing item i ;

 Remove all items from $\mathcal{T}_i$ that are lexicographically $\geq i$;

 Remove all infrequent items from $\mathcal{T}_i$;

 Set up pointers for $\mathcal{T}_i$;

 if $(\mathcal{T}_i \neq \phi)$ then *RecursiveGrowthPointers*($\mathcal{T}_i, minsup, P_i$);

 end

end

Figure 4.10: Generic recursive suffix growth with pointers

Thực hiện với pointer và cây mẫu thường xuyên

- Cấu trúc cây mẫu thường xuyên (FP-tree) được thiết kế với mục đích hiệu quả về **lưu trữ**.
- Cấu trúc này có thể dùng để biểu diễn cơ sở dữ liệu các giao dịch thay cho các mảng.
- Ở thực hiện này của thuật toán, mảng được thay bằng các FP-tree nhưng các pointer vẫn được sử dụng.

Thực hiện với pointer và cây mẫu thường xuyên

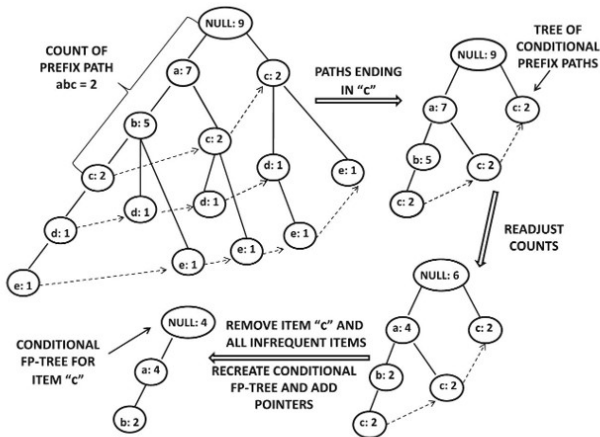


Figure 4.11: Illustration of recursive pattern growth with pointers and FP-Tree

Thực hiện với pointer và cây mẫu thường xuyên

Algorithm *FP-growth*(FP-Tree of frequent items: $\mathcal{FPT}$, Minimum Support: $minsup$, Current Suffix: P)

```
begin
  if  $\mathcal{FPT}$  is a single path
    then determine all combinations  $C$  of nodes on the
         path, and report  $C \cup P$  as frequent;
  else (Case when  $\mathcal{FPT}$  is not a single path)
    for each item  $i$  in  $\mathcal{FPT}$  do begin
      report itemset  $P_i = \{i\} \cup P$  as frequent;
      Use pointers to extract conditional prefix paths
        from  $\mathcal{FPT}$  containing item  $i$ ;
      Readjust counts of prefix paths and remove  $i$ ;
      Remove infrequent items from prefix paths and reconstruct
        conditional FP-Tree  $\mathcal{FPT}_i$ ;
      if ( $\mathcal{FPT}_i \neq \phi$ ) then FP-growth( $\mathcal{FPT}_i, minsup, P_i$ );
    end
end
```

Figure 4.12: The *FP-growth* algorithm with an FP-Tree representation of the transaction database expressed in terms of frequent 1-items

Trade-off giữa các cấu trúc dữ liệu khác nhau

- Về yêu cầu lưu trữ, thực hiện thuật toán với FP-tree nhẹ hơn thực hiện với pointer.
- Tuy nhiên, thực hiện với FP-tree cũng có thể nặng lưu trữ hơn thực hiện với mảng.
- Vấn đề hạn chế trong thực tế thường là việc bộ nhớ có chứa đủ cơ sở dữ liệu giao dịch hay không.

Các phương pháp phát triển mẫu dựa vào hậu tố đệ quy

- Quan hệ giữa phương pháp phát triển mẫu thường xuyên và cây đánh số.
- Phương pháp phát triển cây mẫu thường xuyên có sự tương ứng với phương pháp cây đánh số.

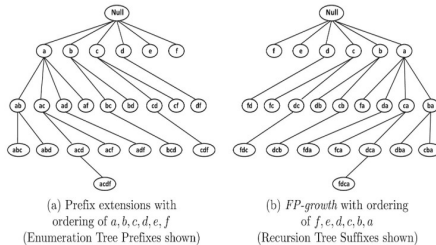


Figure 4.13: Enumeration trees are identical to FP -growth recursion trees with reverse lexicographic ordering

Các phương pháp khác

- Mô hình truyền thống cho việc tìm tập hạng mục thường xuyên được sử dụng rộng rãi do tính đơn giản.
- Sự đơn giản quan trọng này nằm ở:
 - Sử dụng tần suất đơn thuần cho khi tính support các tập hạng mục.
 - Sử dụng xác suất điều kiện để tính độ tin cậy.
- Trong một số ứng dụng, chúng ta muốn có các độ đo khác thích hợp hơn.

Hệ số thống kê cho sự tương quan

Hệ số tương quan Pearson giữa hai biến ngẫu nhiên X và Y được định nghĩa như sau:

$$\rho = \frac{E[X \cdot Y] - E[X] \cdot E[Y]}{\sigma(X) \cdot \sigma(Y)}$$

Trong dữ liệu giỏ hàng, X và Y là các biến nhị phân (có hoặc không xuất hiện sản phẩm). Khi đó, tương quan giữa hai mặt hàng i và j có thể ước lượng bằng:

$$\rho_{ij} = \frac{\text{sup}(\{i, j\}) - \text{sup}(i) \cdot \text{sup}(j)}{\sqrt{\text{sup}(i) \cdot \text{sup}(j) \cdot (1 - \text{sup}(i)) \cdot (1 - \text{sup}(j))}}$$

Hệ số $\rho_{ij} \in [-1, 1]$:

- $+1$: tương quan dương hoàn toàn (*perfect positive correlation*)
- -1 : tương quan âm hoàn toàn (*perfect negative correlation*)
- 0 : không tương quan

Lưu ý: Dễ sai lệch với các item có support nhỏ.

Độ đo χ^2

- Độ đo χ^2 là một độ đo khác xem sự có mặt và vắng mặt của các hạng mục một cách tương tự.
- Cho X là một tập gồm k biến ngẫu nhiên nhị phân (đại diện cho các hạng mục).
- Với O_i và E_i lần lượt là giá trị quan sát được và giá trị kì vọng của support tuyệt đối của trạng thái i :

$$\chi^2(X) = \sum_{i=1}^{2^{|X|}} \frac{(O_i - E_i)^2}{E_i}$$

Tỉ lệ interest

Interest Ratio của tập mục $\{i_1, \dots, i_k\}$ được định nghĩa như sau:

$$I(\{i_1, \dots, i_k\}) = \frac{\sup(\{i_1, \dots, i_k\})}{\prod_{j=1}^k \sup(i_j)}$$

Ý nghĩa:

- Nếu các item độc lập thống kê: $I = 1$
- $I > 1$: các item tương quan dương
- $I < 1$: các item tương quan âm

Lưu ý:

- Dễ gây hiểu lầm với item hiếm xuất hiện
- Không thỏa mãn tính chất **downward closure** → khó thiết kế thuật toán hiệu quả

Độ đo sự tin cậy đối xứng

- Độ đo sự tin cậy truyền thống không đối xứng giữa tiền kiện và hệ quả, trong khi độ đo support đối xứng.
- Chúng ta có thể thay độ đo sự tin cậy này bằng một độ đo đối xứng.

Hệ số cosine trên các cột

The **Cosine coefficient** is usually applied to the rows to determine the similarity among transactions. However, it can also be applied to the columns, to determine the similarity between items.

The cosine coefficient is best computed using the vertical tid list representation on the corresponding binary vectors.

The cosine value on the binary vectors computes to the following:

$$\text{cosine}(i, j) = \frac{\text{sup}(\{i, j\})}{\sqrt{\text{sup}(i)} \cdot \sqrt{\text{sup}(j)}}$$

The numerator can be evaluated as the length of the intersection of the *tid* lists of items *i* and *j*. The cosine measure can be viewed as the geometric mean of the confidences of the rules $\{i\} \Rightarrow \{j\}$ and $\{j\} \Rightarrow \{i\}$. Therefore, the cosine is a kind of **symmetric confidence measure**.

Hệ số Jaccard và kỹ thuật Min-hash

The **Jaccard coefficient** measures similarity between sets. The *tid* lists on a column can be viewed as a set, and the Jaccard coefficient between two *tid* lists can be used to compute the similarity.

Let S_1 and S_2 be two sets. The Jaccard coefficient $J(S_1, S_2)$ between the two sets can be computed as follows:

$$J(S_1, S_2) = \frac{|S_1 \cap S_2|}{|S_1 \cup S_2|}$$

The Jaccard coefficient can easily be generalized to multiway sets, as follows:

$$J(S_1 \dots S_k) = \frac{|\cap S_i|}{|\cup S_i|}$$

Collective strength

The **collective strength** of an itemset is defined in terms of its *violation rate*. An itemset I is said to be in *violation* of a transaction, if some of the items are present in the transaction, and others are not. The *violation rate* $v(I)$ of an itemset I is the fraction of violations of the itemset I over all transactions. The collective strength $C(I)$ of an itemset I is defined in terms of the violation rate as follows:

$$C(I) = \frac{1 - v(I)}{1 - E[v(I)]} \cdot \frac{E[v(I)]}{v(I)}$$

Collective strength

Intuitively, if the violation of an itemset in a transaction is a “bad event” from the perspective of trying to establish a high correlation among items, then $v(I)$ is the fraction of bad events, and $(1 - v(I))$ is the fraction of “good events.” Therefore, collective strength may be understood as follows:

$$C(I) = \frac{\text{Good Events}}{\mathbb{E}[\text{Good Events}]} \cdot \frac{\mathbb{E}[\text{Bad Events}]}{\text{Bad Events}}.$$

Collective strength

The concept of collective-strength may be strengthened to *strongly collective* itemsets.

Definition An itemset I is denoted to be *strongly collective* at level s , if it satisfies the following properties:

1. The collective strength $C(I)$ of the itemset I is at least s .
2. **Closure property:** The collective strength $C(J)$ of every subset J of I is at least s .

Quan hệ với khai phá mẫu negative

- Trong một số ứng dụng, chúng ta muốn xác định mẫu giữa các hạng mục hoặc giữa sự vắng mặt của các hạng mục.
- Các ứng dụng khai phá như cần độ đo đối xứng để xử lý sự có mặt hay vắng mặt của các hạng mục như nhau.
- Một số độ đo như hệ số Jaccard và collective strength thỏa tính chất downward closure có thể được sử dụng cho việc này.

Một số thuật toán meta có ích

- Một thuật toán meta là một thuật toán có sử dụng một thuật toán nào đó bên trong dưới dạng một subroutine.
- Việc này có thể giúp thuật toán chính hiệu quả hơn hoặc có thêm hiểu biết mới.
- Có 2 loại thuật toán meta thường dùng trong khai phá mẫu:
 - Loại sử dụng việc lấy mẫu (sampling) để cải thiện hiệu năng.
 - Loại sử dụng các subroutine tiền xử lý hoặc hậu xử lý để áp dụng vào các trường hợp khác.

Các phương pháp lấy mẫu

- Khi cơ sở dữ liệu giao dịch quá lớn, không thể chứa trong bộ nhớ chính thì chúng ta có thể dùng các phương pháp lấy mẫu.
- Khi dùng thuật toán khai phá tập hạng mục với dữ liệu được lấy mẫu, chúng ta cần quan tâm 2 thử thách sau:
 - **False positive:** các mẫu thỏa trên dữ liệu được lấy mẫu nhưng không thỏa trên dữ liệu gốc.
 - **False negative:** các mẫu không thỏa trên dữ liệu được lấy mẫu nhưng thỏa trên dữ liệu gốc.

Ensemble phân hoạch dữ liệu

- Một cách tiếp cận đảm bảo được không có false positive và false negative là sử dụng các ensemble được phân hoạch.
- Cách này có thể được dùng để giảm tổn kém truy xuất lưu trữ hoặc giảm tổn kém bộ nhớ.

Tổng quát hóa đến các kiểu dữ liệu khác

Việc tổng quát hóa đến các kiểu dữ liệu khác có thể được thực hiện với các phương pháp biến đổi kiểu dữ liệu.

- 1 Các thuật toán khai phá tập hạng mục thường xuyên
 - Các phương pháp phát triển mẫu dựa vào hậu tố đệ quy
 - Các phương pháp khác
 - Một số thuật toán meta có ích
- 2 Phân tích gom nhóm
 - Gom nhóm
 - Chọn đặc trưng để gom nhóm

Một số ứng dụng tiêu biểu của việc gom nhóm:

- Tóm tắt dữ liệu.
- Chia nhóm khách hàng.
- Phân tích mạng xã hội.
- Hỗ trợ các ứng dụng khai phá dữ liệu khác.

Chọn đặc trưng để gom nhóm

- Mục tiêu chính của việc chọn đặc trưng là để loại bỏ các thuộc tính nhiễu.
- Việc chọn lọc đặc trưng thường khó hơn với các bài toán không giám sát như gom cụm do thiếu các tiêu chí đánh giá ngoài như nhãn dữ liệu.
- Có 2 lớp mô hình chính để chọn đặc trưng
 - Mô hình lọc.
 - Mô hình gói.

- Với các mô hình lọc, một tiêu chí cụ thể được dùng để đánh giá ảnh hưởng của các đặc trưng hoặc nhóm đặc trưng cụ thể.
- Chúng ta có một số tiêu chí phổ biến như sau:
 - Term strength.
 - Sự phụ thuộc thuộc tính dự đoán.
 - Entropy.
 - Thống kê Hopkins.

Term strength

- Term strength is suitable for sparse domains like text data, where presence/absence of attributes (words) is more meaningful than distances.
- Similarity functions are preferred over distance functions in such domains.
- Term strength is calculated by sampling document pairs with a random ordering. It is defined as:

$$\text{Term Strength} = P(t \in \overline{Y} \mid t \in X),$$

where $(X, \overline{Y})$ are similar document pairs (similarity $> \beta$), and t occurs in both documents.

- For multidimensional data, term strength can be generalized by discretizing quantitative attributes into binary values.
- Other relevance measures use correlations between overall distances and attribute-wise distances.

Sự phụ thuộc thuộc tính dự đoán

- Correlated features will always result in better clusters than uncorrelated features.
- When an attribute is relevant, other attributes can be used to predict the value of this attribute:
 - For numeric attributes, use regression modeling.
 - For categorical attributes, use classification algorithms.
- The overall approach for quantifying the relevance of an attribute i is as follows:
 1. Use a classification algorithm on all attributes, except attribute i , to predict the value of attribute i , while treating it as an artificial class variable.
 2. Report the classification accuracy as the relevance of attribute i .
- Any reasonable classification algorithm can be used, although a nearest neighbor classifier is desirable because of its natural connections with similarity computation and clustering.

Entropy

- Ý tưởng chính ở đây là việc các dữ liệu có nhóm thể hiện các đặc trưng gom nhóm trên phân phối khoảng cách tương ứng.
- Mục tiêu của các độ đo entropy là định lượng hóa hình dạng của phân phối khoảng cách với một tập con các đặc trưng.

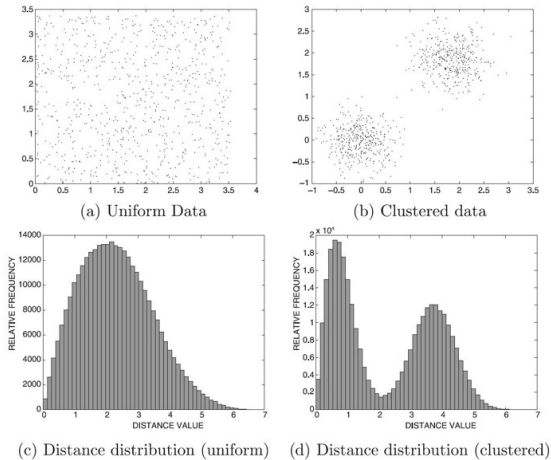


Figure 6.1: Impact of clustered data on distance distribution entropy

Thống kê Hopkins

Let D be the data set whose clustering tendency needs to be evaluated. A sample S of r synthetic data points is randomly generated in the domain of the data space. At the same time, a sample R of r data points is selected from D .

Let $\alpha_1 \dots \alpha_r$ be the distances of the data points in the sample $R \subseteq D$ to their nearest neighbors within the original database D . Similarly, let $\beta_1 \dots \beta_r$ be the distances of the data points in the synthetic sample S to their nearest neighbors within D .

Then, the Hopkins statistic H is defined as follows:

$$H = \frac{\sum_{i=1}^r \beta_i}{\sum_{i=1}^r (\alpha_i + \beta_i)}.$$

The Hopkins statistic will be in the range $(0, 1)$.

- Uniformly distributed data will have a Hopkins statistic of 0.5 because the values of α_i and β_i will be similar.
- For clustered data, the values of α_i will typically be much lower than β_i . This will result in a value of the Hopkins statistic that is closer to 1.

Therefore, a high value of the Hopkins statistic H is indicative of highly clustered data points.

Mô hình gói

- Các mô hình gói sử dụng một tiêu chí đánh giá gom nhóm bên trong cùng với một thuật toán gom nhóm áp dụng lên một tập các đặc trưng thích hợp.
- Ý tưởng ở đây là dùng một thuật toán gom nhóm với một tập các đặc trưng và sau đó đánh giá chất lượng với tiêu chí được chọn.
- Một khuyết điểm lớn của cách này là sự nhạy với tiêu chí đánh giá được chọn.

Một cách khác đơn giản hơn là chọn các đặc trưng riêng lẻ với một tiêu chí chọn đặc trưng của các thuật toán phân loại:

- Use a clustering algorithm on the current subset of selected features F , in order to fix cluster labels L for the data points.
- Use any *supervised* criterion to quantify the quality of the individual features with respect to labels L . Select the top- k features on the basis of this quantification.

Thuật toán Generic Representative

Algorithm *GenericRepresentative*(Database: $\mathcal{D}$, Number of Representatives: k)
begin
 Initialize representative set S ;
 repeat
 Create clusters $(\mathcal{C}_1 \dots \mathcal{C}_k)$ by assigning each
 point in $\mathcal{D}$ to closest representative in S
 using the distance function $Dist(\cdot, \cdot)$;
 Recreate set S by determining one representative $\bar{Y}_j$ for
 each $\mathcal{C}_j$ that minimizes $\sum_{\bar{X}_i \in \mathcal{C}_j} Dist(\bar{X}_i, \bar{Y}_j)$;
 until convergence;
 return $(\mathcal{C}_1 \dots \mathcal{C}_k)$;
end

Figure 6.2: Generic representative algorithm with unspecified distance function